

Experiment No.08

Aim: Enable real-time communication via WebSockets.

Source Code:

server.js

```
1  const express = require("express");
2  const http = require("http");
3  const { Server } = require("socket.io");
4  const path = require("path");
5
6  const app = express();
7  const server = http.createServer(app);
8  const io = new Server(server);
9
10 app.use(express.static(path.join(__dirname, "../frontend")));
11
12 io.on("connection", (socket) => {
13   console.log("🟢 New user connected");
14
15   socket.on("new_user", (username) => {
16     socket.username = username;
17     io.emit("receive_message", { username: "System", message: `🟢 ${username} joined the chat` });
18   });
19
20   socket.on("send_message", (data) => {
21     socket.broadcast.emit("receive_message", data);
22   });
23
24   socket.on("disconnect", () => {
25     if (socket.username) {
26       io.emit("receive_message", { username: "System", message: `🔴 ${socket.username} left the chat` });
27     }
28   });
29 });
30
31 const PORT = 5000;
32 server.listen(PORT, () => console.log(`🚀 Server running on http://localhost:${PORT}`));
33
```

chat.js

```
1  const socket = io();
2
3  let username = prompt("Enter your name:");
4  if (!username || username.trim() === "") username = "Anonymous";
5
6  document.getElementById("userDisplay").textContent = `You: ${username}`;
7
8  socket.emit("new_user", username);
9
10 const form = document.getElementById("chatForm");
11 const input = document.getElementById("messageInput");
12 const chatBox = document.getElementById("chatBox");
13
14 form.addEventListener("submit", (e) => {
15   e.preventDefault();
16   const message = input.value.trim();
17   if (message) {
18     socket.emit("send_message", { username, message });
19     appendMessage("You", message, "right");
20     input.value = "";
21   }
22 });
23
24 socket.on("receive_message", (data) => {
25   if (data.username === "System") {
26     const msg = document.createElement("p");
27     msg.className = "system";
28     msg.textContent = data.message;
29     chatBox.appendChild(msg);
30   } else if (data.username !== username) {
31     appendMessage(data.username, data.message, "left");
32   }
33   chatBox.scrollTop = chatBox.scrollHeight;
34 });
35
36 function appendMessage(name, message, position) {
37   const msg = document.createElement("div");
38   msg.classList.add("message", position);
39   msg.textContent = `${name}: ${message}`;
40   chatBox.appendChild(msg);
41 }
42
```

Index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Realtime Group Chat</title>
7   <link rel="stylesheet" href="css/style.css" />
8   <script src="https://cdn.socket.io/4.7.2/socket.io.min.js"></script>
9 </head>
10 <body>
11   <div class="container">
12     <div class="header">
13       🗨 Group Chat App
14       <span id="userDisplay" class="username-tag"></span>
15     </div>
16
17     <div class="chat-box" id="chatBox"></div>
18
19     <form id="chatForm">
20       <input id="messageInput" placeholder="Type a message..." autocomplete="off" />
21       <button type="submit">Send</button>
22     </form>
23   </div>
24   <script src="js/chat.js"></script>
25 </body>
26 </html>
27
```

Output:



